

Assignment 5 Report

Software Development for Robotics

Group: PPD2

V. Malaxa s2726254, R. Díaz s3681548

March 23, 2026

5.1.1 Block diagram

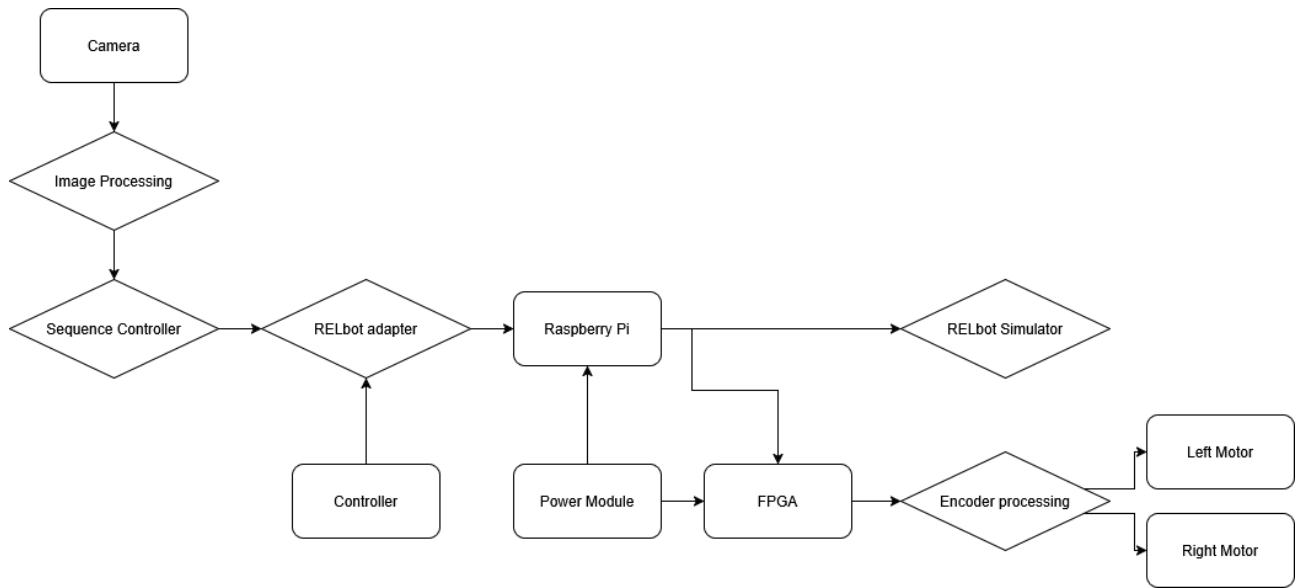


Figure 1: High level Block Diagram of the RELbot. Squares represent the hardware of the robot, while the diamonds represent the software components.

Table 1: Components of the RELbot system

Blocks	Type	Description
Camera	Physical	Hardware sensor that captures images of the environment. It provides a continuous stream of visual data (camera frames) that are used for perception and navigation.
Image Processing	Software	Software component (ROS2 node) that processes raw camera images to extract meaningful information, such as detecting objects and determining their position in the image. It converts image data into structured information usable for control.
Sequence controller	Software	High-level control component (ROS2 node) that interprets processed image data and determines the desired robot behavior. It generates motion commands (e.g. linear and angular velocity) based on the detected object or desired trajectory.
RELbot adapter	Software	Interface component that receives motion commands from the Sequence Controller and converts them into safe and valid control signals for the robot. It limits the commands to safe ranges and routes them either to the simulator or to the real robot hardware.
Controller	Physical	External input device (e.g. joystick or keyboard) that can generate manual motion commands. It can override or complement the automatic control in situations such as testing or when the robot needs manual intervention.
Raspberry Pi	Physical	Embedded computer that runs the ROS2 software stack. It handles communication between software nodes, processes incoming commands, and forwards control signals to the low-level real-time system (FPGA).
FPGA	Physical	Hardware component responsible for low-level, real-time control of the motors. It receives control commands (e.g. wheel velocities) and converts them into electrical signals that drive the actuators.
Power module	Physical	Provides electrical power to all components of the system, including the Raspberry Pi, FPGA, sensors, and motors. Ensures stable operation of the robot.
RELbot Simulator	Software	Software model of the robot used for testing and development. It simulates the robot's motion and sensor outputs, allowing the same control software to be tested without using the physical robot.
Encoder processing	Software	Software component that processes encoder feedback signals from the motors. It converts raw encoder data into wheel position or velocity.
Motors	Physical	Actuators that drive the robot's wheels. They convert electrical control signals into mechanical motion, enabling the robot to move through the environment.

Table 2: Signals of the RELbot system

Signal name	From → To	Unit	Description
/image_raw	Camera → Image Processing	pixels	Raw camera image stream
/object_position	Image Processing → Sequence Controller	m	Detected object position
/cmd_vel	Sequence Controller → RELbot Adapter	m/s, rad/s	Desired robot velocity
/validated_cmd	RELbot Adapter → Robot/Simulator	m/s, rad/s	Limited/safe velocity command
/joy_cmd	Controller → RELbot Adapter	m/s, rad/s	Manual control input
/motor_cmd	RELbot Adapter → FPGA	rad/s or PWM	Low-level motor commands
/encoder_data	FPGA → Encoder Processing	ticks / rad	Encoder feedback from motors
/angle	Encoder Processing → Motors	rad	Wheel angle or position feedback
/power	Power Module → Raspberry Pi, FPGA	V	Power supply to system

5.3.1 Testing of provided code

Before programming the sequence controller, it was useful to understand the already available topics inside the provided workspace for this assignment by using "ros2 topic list". After this, info on the "motor_cmd" was obtained via the "ros2 topic info" command, and finally using "ros2 interface show", the message structure in charge of controlling the motors was inspected, figure [2] displaying the output obtained:

```
ricardodiaz@DESKTOP-UH3E0CJ:~$ ros2 interface show relbot_msgs/msg/RelbotMotors
# Unified motor velocities, rather than two possible outputs of sync topics
float64 left_wheel_vel
float64 right_wheel_vel
```

Figure 2: Terminal output showing the messages in RelbotMotors, in charge of updating the speed of the ReLbot wheels.

Having observed how these two topics control the speeds of the two wheels of the ReLbot (or in the case of simulation, the turtle), testing with the terminal was carried out to determine how each of these two topics affected movement of the turtle by using "ros2 topic pub" on the relevant topics. Figure [3] below shows the testing for straight movement of the turtle.

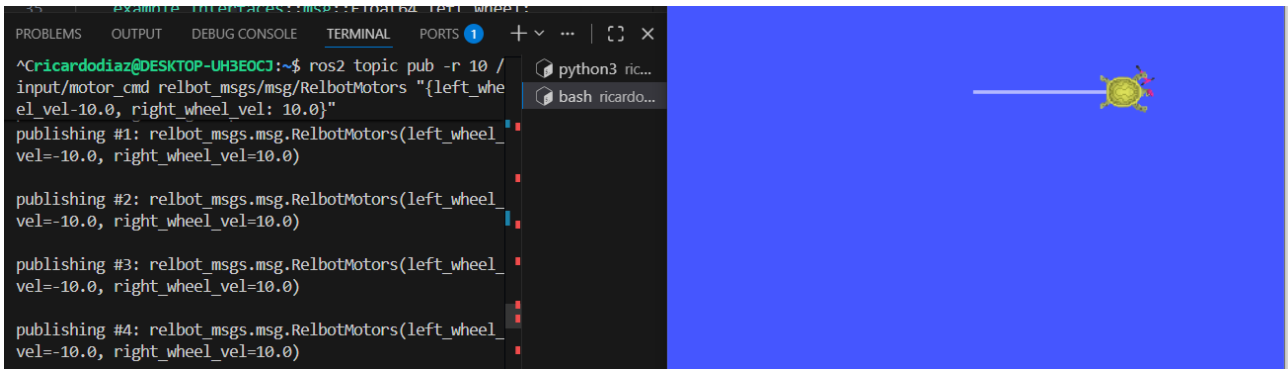


Figure 3: Terminal output with the result of publishing specific velocities in the ReLbot wheels using turtlesim (right) to simulate the movement.

5.3.2 Sequence controller Design Choices

After testing the control of the turtle using the terminal, the file "relbot_sequence_controller.cpp" was edited to include all of the described trajectories on the assignment guide. The first step is initializing the "selection" variable (line 38) with the desired value, which controls which mode the robot will move in. The function call `get_clock()` is used in order to obtain the current ROS2 system time at the moment of starting execution. After this, an if statement is used to reset the timer. This was implemented to avoid large inconsistencies in trajectories due to leftover timing from previous trajectories.

After this security measure, the elapsed time is computed by subtracting the current time obtained using `get_clock()` with the variable storing the starting time, obtaining our "t" variable which will be used for timing of the movements of the ReLbot.

Each of the needed trajectories is executed by updating the speed on each of the wheels (right and left) of the robot, and which of the trajectories to follow is determined by an if statement based on the value of the "selection" int variable:

- If **selection = 1**, the followed trajectory will be a straight line. This is programmed simply by changing the speed of each of the wheels to the same absolute value. In the ReLbot, left and right wheel commands seem to have opposite convention, so the left wheel having a negative value while the right leg keeps the inverted positive value means that both wheels will move forward at the same speed preventing any rotation. In the case that it is needed to increase the speed at which the robot moves, simply setting both the left and right speed to bigger absolute values will result in faster speeds. After 5 seconds moving straight, the robot stops moving by using an else if statement with both the specific selection and exceeding the maximum time as conditions to stop the robot. The resulting shape can be seen in Figure [4], and is a 50 centimeter line assuming [cm/s] units for the speed of the ReLbot, as it takes 5 seconds moving at a speed of 10 cm/s.

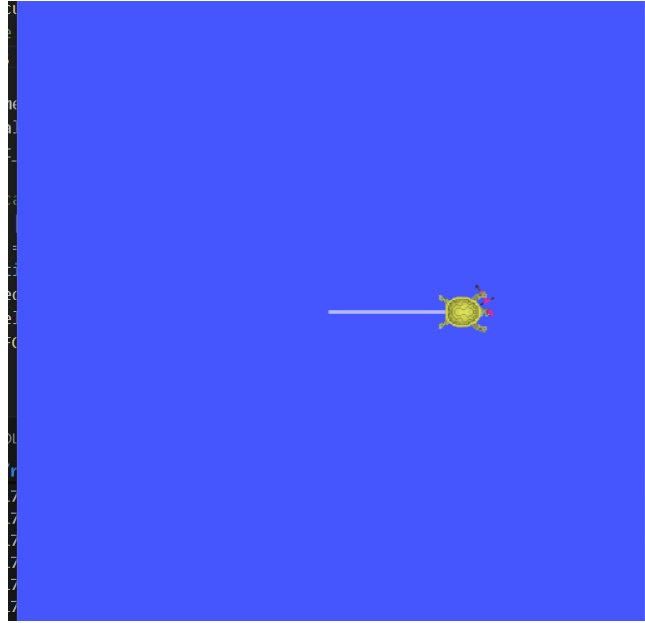


Figure 4: Straight trajectory followed via turtlesim simulation.

- If **selection** = 2, the followed trajectory will be a circle. For this, the magnitude of one of the two wheels must be smaller than that of the other wheel. The smaller the difference in magnitude, the bigger the radius of the resulting circle will be. The chosen values for this circle trajectory were -7.5 for the left wheel speed and 10 for the right wheel speed. This resulted in the circle shown by Figure [5], with the robot stopping after 15 seconds of following the circular trajectory, by using the same type of else if statement as described in the previous case.

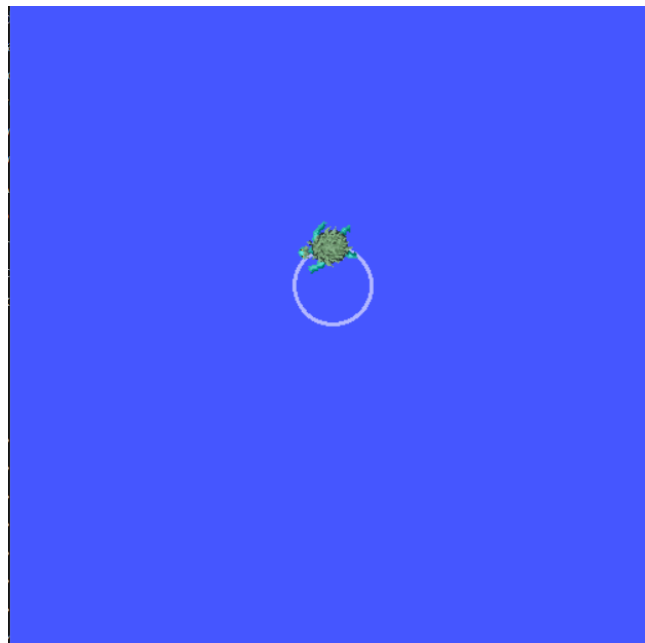


Figure 5: Circle trajectory followed via turtlesim simulation.

The circle drawn using this parameters has a radius which can be computed in terms of the distance between the two wheels of the robot (L) as specified in the equations below.

The speeds are assumed to be in cm/s.

$$v = \frac{v_r + v_l}{2} = 8.75 \quad (1)$$

$$\omega = \frac{v_r - v_l}{L} = \frac{2.5}{L} \quad (2)$$

$$R = \frac{v}{\omega} = \frac{L}{2} \cdot \frac{v_r + v_l}{v_r - v_l} = 3.5L[cm] \quad (3)$$

- If **selection** = 3, the followed trajectory will be a straight line, followed by a 90 degrees turn and another straight line. For this case, first an if statement checks whether the selection variable is at 3 and as long as the time is less than 5 seconds, the robot continues in a straight line. After 5 seconds, the original if statement is no longer valid ($t > 5s$) and a different else if is used to set the speed of the left wheel to 0 while the right wheel advances with a speed magnitude of 10 leading to a rotation in place. After 0.635 seconds, the else if statement is no longer valid ($t > 5.635$) and the next else if is fulfilled, changing the magnitude of the speed of both wheels so that they are equal (left = -10 and right = 10). After this the robot starts moving in a straight line again and 5 more seconds, the robot is stopped by jumping to a new else if where the speeds are both set to 0. The 0.635 seconds used to make the sharp 90 degrees turn were obtained via trial and error, observing how different timings affected the angle of the turn, until 0.635 seconds of the turning movement were determined to provide a satisfying angle. The resulting shape can be observed in Figure[6] and has two lines of 50 cm as it takes 5 seconds at a speed of 10 cm/s to draw each line:

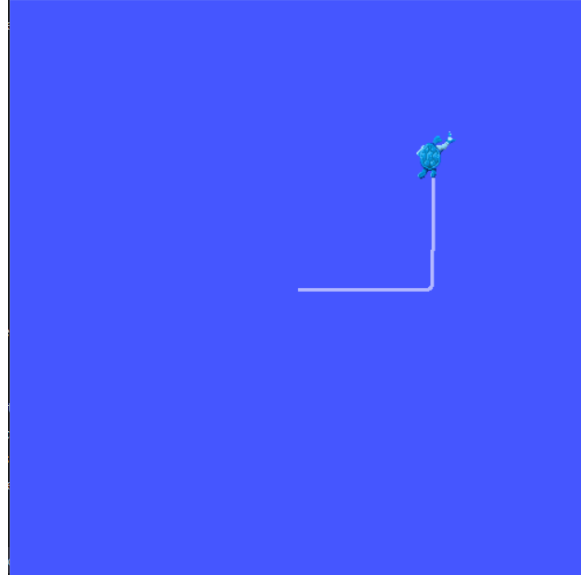


Figure 6: Sharp turn trajectory followed via turtlesim simulation.

- If **selection** = 4, the followed trajectory will be a square. This square trajectory was built by chaining together the chain of else if statements used for the previously described sharp turn trajectory four times. In order for this to work, the time limits on each of the else if statements had to be changed so that all sides were of the same length and the timing for the turns was correct. Each straight line took 5 seconds using speeds with a magnitude of 10, and each sharp turn took 0.635 seconds, with the left wheel having a speed of 0 and the right wheel having a speed of 10. Since each of the sides took 5 seconds using a speed with a magnitude of 10, one can compute that the created shape

results in a square with sides of 50 centimeters (assuming velocity is expressed in [cm/s]). Figure [7] displays the resulting shape. It is worth mentioning that each time the sequence controller was executed with this trajectory selected, the shape of the square varied slightly. This could be explained by small variations in the system time obtained by ROS2.

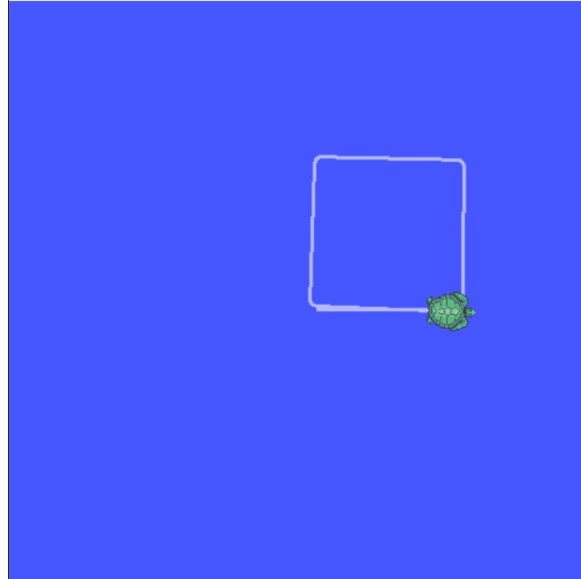


Figure 7: Square trajectory followed via turtlesim simulation.

- If selection is **different from the expected 4 options**, an error message will continuously be displayed in the terminal by using the `RCLCPP_ERROR()` function informing the user about the use of an invalid selection

The use of only "if" and "else if" statements for the logic of the trajectories of this assignment can be justified by the simplicity of the trajectories modeled, but in the case of a more complex movement being required, using switch statements as well as for and while loops may be a better option, as it will make for a more readable and compact code and decrease the complexity of some scenarios.

Regarding testing for this assignment, the only scenario where there could be unexpected error was covered with the inclusion of an error message in the case that the user selects an invalid value for the selection int. The code was tested multiple times in each of the trajectories to ensure an acceptable desired shape was obtained in each of them consistently.

1 AI statement

During the writing of this assignment AI tools(such as ChatGPT) were only used to identify errors in the code and format the Latex document. After which the code was reevaluated and rewritten by the user themselves.